Calling REST APIs

Module 4 - Lesson 2

Overview

REST APIs let programs exchange data over HTTP. The requests library turns a URL into typed Python data, and understanding status codes keeps calls robust.

Key Concepts

- HTTP methods map to actions: GET reads, POST creates, PUT/PATCH update, DELETE removes.
- requests.get(url) returns a Response; .status_code and .json() read it.
- Pass parameters with params=, headers with headers=, and bodies with json=.
- Handle errors: raise_for_status() raises for 4xx/5xx responses.
- Add timeouts so a slow API cannot hang your program forever.
- Respect APIs: use auth tokens, handle rate limits, and retry transient failures.

Example

```
import requests

res = requests.get(
    "https://api.github.com/repos/python/cpython/issues",
    params={"state": "open", "per_page": 5},
    timeout=10,
)
res.raise_for_status()
issues = res.json()
print(len(issues), "open issues")
```

Summary

The requests library makes REST calls a few lines of code. Checking status, setting timeouts, and handling errors turn flaky calls into dependable data pipelines.

Practice Checklist

- Fetch data from a public API and print a field from the response.
- Add params and headers to a request and inspect the URL that was sent.
- Wrap a call in try/except and add a timeout.